



Historial y memoria (corta vs “larga”)

Un LLM **no tiene memoria** entre llamadas a la API. Si hoy le dices “Me llamo Ana” y mañana preguntas “¿Cómo me llamo?”, el modelo solo podrá responder si **tú** le vuelves a enviar ese dato como parte del contexto.

Tenemos que distinguir entre:

- **Historial conversacional**: reenviar turnos previos para mantener coherencia.
 - **Memoria corta** (ventana): últimos N turnos / últimos N eventos recientes.
 - **Memoria “larga” simple** (sin embeddings): perfil del usuario o datos persistentes en tu aplicación.
-

Objetivos

- Entender por qué “el modelo no recuerda” y cómo simular memoria reenviando historial.
 - Implementar historial en Python (lista de mensajes o sesión de chat).
 - Construir una “memoria larga” básica con `dict` (perfil/preferencias).
 - Trabajar con `state.py` como estado de sesión. Ahora le daremos más importancia.
-

1) Dos formas de gestionar historial con Gemini

Opción A — Historial explícito (tú lo gestionas)

La API es “stateless” (sin estado): envías `contents` y recibes una respuesta. Si quieres memoria, pasas un **listado** con los turnos.

```
from google import genai
from google.genai import types

MODEL = "gemini-3-flash-preview"
client = genai.Client()

history: list[types.Content] = []

def append_user(texto: str) -> None:
```

```

        history.append(
            types.Content(
                role="user",
                parts=[types.Part.from_text(text=texto)],
            )
        )

def append_model(texto: str) -> None:
    history.append(
        types.Content(
            role="model",
            parts=[types.Part.from_text(text=texto)],
        )
    )

def preguntar(user_text: str) -> str:
    append_user(user_text)
    r = client.models.generate_content(model=MODEL, contents=history)
    respuesta = (r.text or "").strip()
    append_model(respuesta)
    return respuesta

print(preguntar("Me llamo Ana. Recuérdalo."))
print(preguntar("¿Cómo me llamo?"))

```

Ventaja: control total. Inconveniente: tú decides cuándo recortar o resumir.

Opción B — Sesión de chat (el SDK mantiene el historial)

```

from google import genai

MODEL = "gemini-3-flash-preview"
client = genai.Client()

chat = client.chats.create(model=MODEL)
print(chat.send_message("Me llamo Ana. Recuérdalo.").text)
print(chat.send_message("¿Cómo me llamo?").text)

```

Ventaja: menos código para demos. Inconveniente: igualmente necesitas estrategia de recorte cuando crezca.

2) “Memoria corta”: ventana de los últimos N turnos

En apps reales no puedes reenviar una conversación infinita. Una primera estrategia (didáctica) es enviar solo los últimos N turnos.

Si gestionas historial explícito:

```

WINDOW = 10 # últimos 10 mensajes (user/model mezclados)

def history_window() -> list[types.Content]:
    return history[-WINDOW:] # devolvemos los últimos 10 mensajes

def preguntar_con_ventana(user_text: str) -> str:
    append_user(user_text)
    r = client.models.generate_content(model=MODEL, contents=history_window())
    respuesta = (r.text or "").strip()
    append_model(respuesta)
    return respuesta

```

Esto crea un comportamiento típico:

- cosas recientes → el modelo “las recuerda”
- cosas antiguas → se pierden

Ese es el punto: la memoria es una consecuencia de lo que envías.

3) Estado en memoria como base. Uso de `state.py`

Anteriormente ya trabajamos con algo parecido a “memoria de sesión” con `state.py`:

- `state["historial"]` guarda eventos válidos
- `ultimos_n(state, n)` devuelve los últimos elementos

Ese patrón se reutiliza aquí: el historial de tu app (eventos) puede convertirse en “contexto reciente”.

Ejemplo de enfoque (sin API todavía):

```

from state import ultimos_n

def build_recent_events_context(state: dict, n: int = 3) -> str:
    eventos = ultimos_n(state, n)
    if not eventos:
        return ""
    lines = ["Eventos recientes (válidos):"]
    for e in eventos:
        lines.append(f"- {e.get('nombre')} · {e.get('tipo')} · prioridad {e.get('prioridad')}")
    return "\n".join(lines)

```

4) “Memoria larga” simple: perfil del usuario (sin embeddings)

Muchas apps necesitan recordar **preferencias** o datos estables:

- idioma preferido

- nivel ("junior", "intermedio"...)
- tono ("formal", "cercano"...)

Eso no depende del historial reciente. Suele vivir en una estructura persistente:

```
user_profile = {
    "name": "Ana",
    "language": "español",
    "level": "junior",
    "tone": "claro y directo",
}
```

El truco es **inyectarlo** cuando haga falta:

```
def build_profile_context(profile: dict) -> str:
    return (
        "Perfil del usuario:\n"
        f"- Nombre: {profile.get('name')}\n"
        f"- Idioma: {profile.get('language')}\n"
        f"- Nivel: {profile.get('level')}\n"
        f"- Estilo: {profile.get('tone')}\n"
    )

def build_prompt(pregunta: str, profile: dict, recent: str) -> str:
    return f"""
Instrucciones: responde adaptándote al nivel del usuario.

{build_profile_context(profile)}

{("Contexto reciente:\n" + recent) if recent else ""}

Pregunta:
{pregunta.strip()}
""".strip()
```

Esto enseña una idea central: **contexto ≠ conversación**. A veces el contexto es un “estado” externo.

5) Errores comunes

1. **Crear que el modelo “recuerda”** por sí mismo → la API es sin estado.
2. **Reenviar todo siempre** → sube coste/latencia y aumenta ruido.
3. **Guardar texto sin estructura** → difícil recortar o resumir (mejor listas/dicts).
4. **Mezclar perfil con historial** → el perfil es estable; el historial es temporal.

Resumen

- Historial = reenviar turnos; “memoria aparente” = consecuencia del historial.

- Memoria corta = ventana de últimos N turnos/eventos.
- Memoria "larga" simple = perfil/preferencias en `dict` que inyectas cuando aplica.